

SCRUM, GIT, GITHUB AND GITHUB ACTIONS

To-Do List Management System Assignment Report

Student	Kartik R Mahindrakar
Project	Daymark To-Do List Management System
Jira Project	To-Do List Management System (key: TODO)
Sprint	To-Do Sprint 1
Date	8 September 2026
GitHub Repository	https://github.com/kartikm95-2006/taskline-to-do-list

1. Project Overview

Daymark is a browser-based To-Do List Management System built with HTML, CSS, and JavaScript. It runs without package installation or a build step. Users can add tasks, mark tasks completed, filter tasks, delete tasks, view the current date, persist tasks in local storage, and see completion progress.

Technology: HTML, CSS, JavaScript, browser localStorage, Git, GitHub, Jira, and GitHub Actions. The responsive interface is designed for desktop and mobile screens.

2. Jira Scrum Project

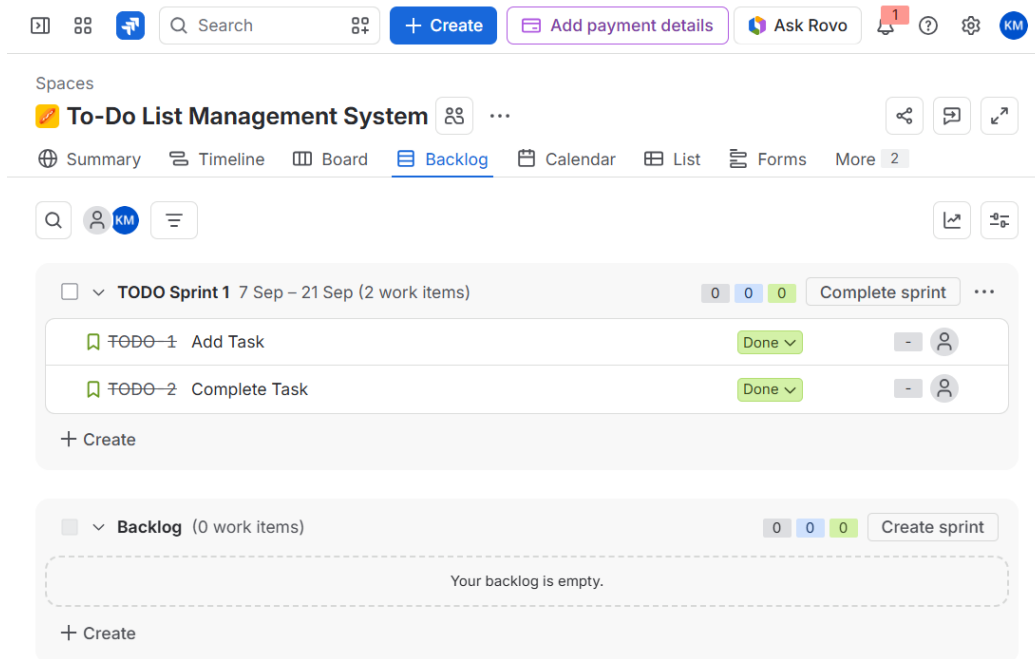
The Jira project is To-Do List Management System with project key TODO. The sprint is To-Do Sprint 1.

User Stories

1. Add Task: As a user, I want to add a new task to my To-Do List so that I can keep track of my work.
2. Complete Task: As a user, I want to mark a task as completed so that I can identify the tasks I have finished.

Jira execution: TODO-1 and TODO-2 were placed in the sprint and progressed through To Do, In Progress, and Done. The final sprint view shows both stories as Done and the separate backlog empty.

Jira evidence: sprint and completed stories



This screenshot shows the TODO project Backlog view, TODO Sprint 1, both stories, their green Done statuses, and an empty separate backlog. It verifies the Scrum sprint setup and final workflow result.

3. Sprint Planning and Execution

Both stories were assigned to To-Do Sprint 1 before execution. The sprint was started for the active two-week window, and each story was moved through To Do, In Progress, and Done. Jira's final sprint summary shows 0 items in To Do, 0 items in In Progress, and 2 items in Done.

Status-history evidence: Jira records the stories being assigned to the sprint and progressing from To Do to In Progress and then to Done. The captured final board is the visible completion evidence for this sequence.

4. Git Repository and Operations

The local repository was initialized, the project files were committed, a feature branch was created and merged into main, and the result was pushed to GitHub.

Command	Purpose
git init	Initializes a new local Git repository.
git status	Displays the current working-tree state.
git add .	Stages all project files for commit.
git commit -m "Initial commit"	Records the staged files in repository history.
git branch feature-task	Creates a feature branch.
git checkout feature-task	Switches to the feature branch.
git checkout main	Switches back to the main branch.
git merge feature-task	Merges the feature branch into main.
git remote add origin	Connects the local repository to GitHub.
git push -u origin main	Publishes main and sets its upstream branch.
git remote -v	Displays the configured fetch and push URLs.
git log --oneline --decorate -3	Shows the latest commits and branch references.

Terminal evidence: complete Git sequence

```
Windows PowerShell | C:\Users\Kartik Mahindrakar\Desktop\To Do List

Git operations completed for Taskline

PS> git init
Initialized empty Git repository
PS> git add .
PS> git commit -m "Initial commit"
[main 6c8e97f] Initial commit
PS> git branch feature-task
PS> git checkout feature-task
Switched to branch 'feature-task'
PS> git checkout main
PS> git merge feature-task
Fast-forward; feature branch merged into main
PS> git remote add origin https://github.com/kartikm95-2006/taskline-to-do-list.git
PS> git push -u origin main
main -> main; branch set up to track origin/main

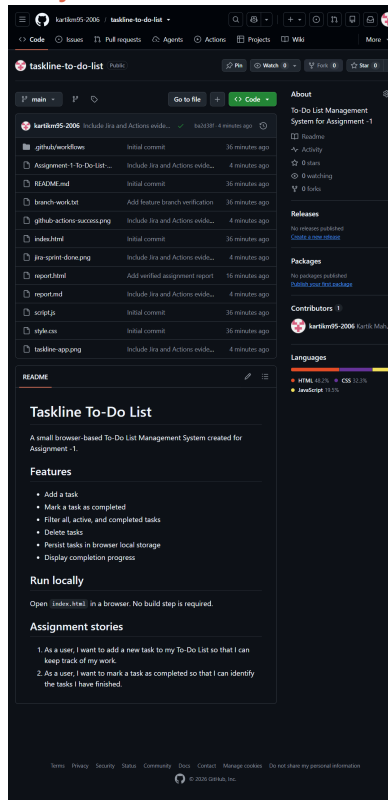
Final verification: ## main...origin/main | working tree clean
```

This screenshot records git init, staging, the initial commit, feature branch creation, checkout, merge, remote configuration, and push. The final verification confirms main tracks origin/main and the working tree is clean.

5. GitHub Repository

The published repository is <https://github.com/kartikm95-2006/taskline-to-do-list>. The main branch contains the application, README, report, evidence, and workflow files.

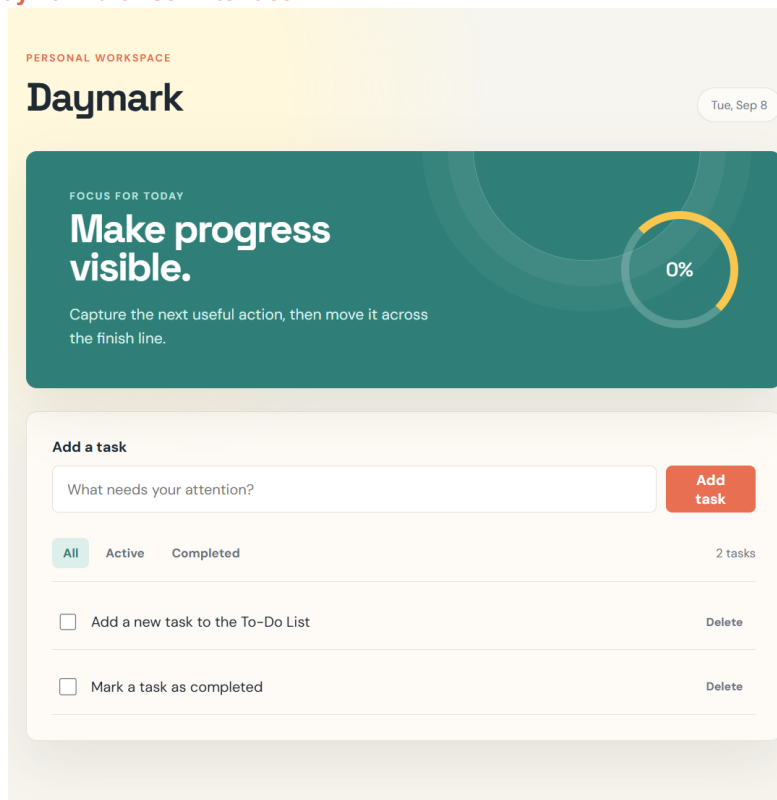
GitHub website evidence: published repository



This screenshot shows the repository under the kartikm95-2006 account. It verifies that the project files and assignment report were published to GitHub.

6. Delivered Application

Application evidence: Daymark browser interface



The application screenshot displays two assignment-related open tasks and reports 2 tasks. This confirms that client-side task creation, persistence, filtering, and open-task counting are functioning.

7. GitHub Actions

The workflow file is `.github/workflows/workflow.yml` and runs automatically on pushes to main and pull requests targeting main. It also supports manual execution. The validation job checks `index.html`, `style.css`, `script.js`, the Daymark title, and the task form.

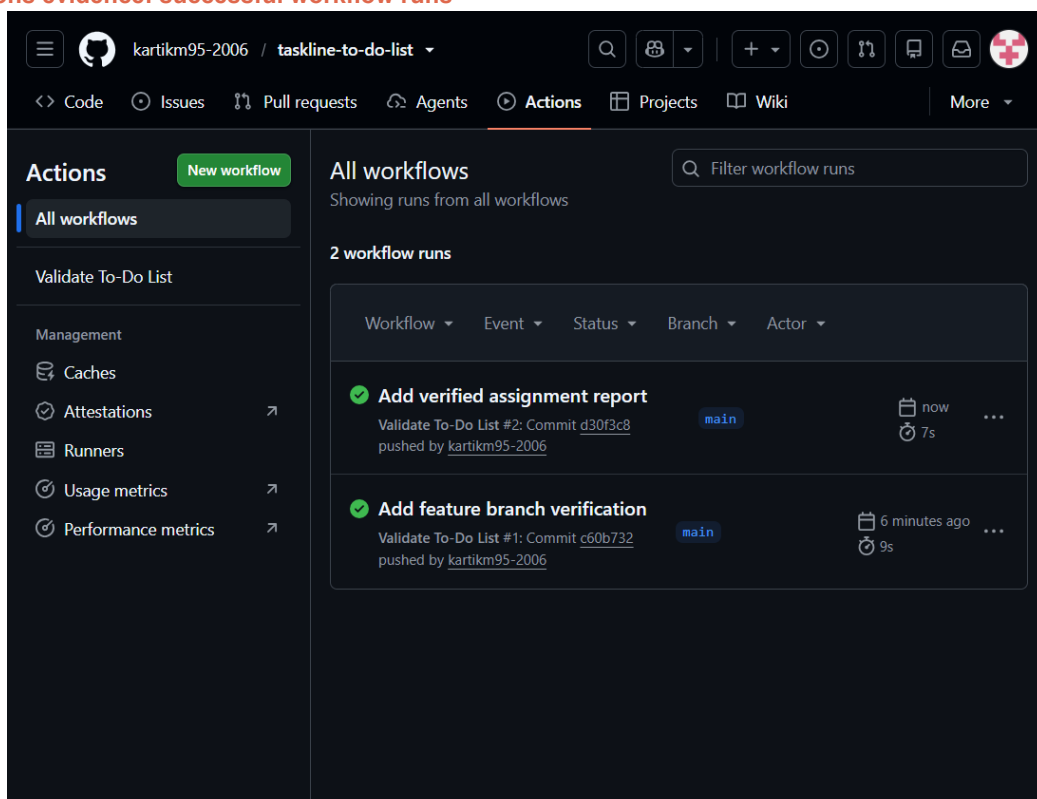
```
name: Daymark To-Do List CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v4
      - name: Validate project files
        run: |
          test -f index.html
          test -f style.css
          test -f script.js
          grep -q "Daymark" index.html
          grep -q "taskForm" script.js
          echo "Daymark To-Do List project files validated successfully"
```

GitHub Actions evidence: successful workflow runs



The green checks show successful Daymark To-Do List CI runs on main. The workflow executes after pushes and is configured to validate pull requests targeting main as well.

8. Conclusion

The Daymark To-Do List Management System was planned with two Jira Scrum stories, delivered through one sprint, versioned with Git, published to GitHub, and validated automatically with GitHub Actions. The repository link and

evidence are included for submission.